

Week-2:

Task-1:

```
GNU nano 8.7.1 week1practical.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    char command[100];
    pid_t pid;

    printf("Enter a Linux command: ");
    scanf("%s", command);

    pid = fork();

    if(pid < 0)
    {
        printf("Fork Failed!\n");
        return 1;
    }

    else if(pid == 0)
    {
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("\nExecuting command: %s\n", command);

        execlp(command, command, NULL);

        printf("Command execution failed.\n");
    }

    else
    {
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID : %d\n", pid);

        wait(NULL);

        printf("\nChild process completed.\n");
    }

    return 0;
}
```

GNU nano 8.7.1 week1practical.c *
^G Help ^O Write Out ^W Where Is ^X Cut ^T Execute ^C Location ^H-U Undo ^M-A Set Mark ^M-J To Bracket ^M-< Previous
^X Exit ^R Read File ^E Replace ^U Paste ^J Justify ^_ Go To Line ^H-G Redo ^M-G Copy ^M-W Where Was ^M-> Next

```
samanth@samanth-ubuntu: ~  
samanth@samanth-ubuntu:~$ nano task1.c  
samanth@samanth-ubuntu:~$ gcc task1.c -o task1  
samanth@samanth-ubuntu:~$ ./task1  
Enter a Linux command: ls  
  
Parent Process  
Parent PID: 3142  
Child PID : 3143  
  
Child Process  
Child PID : 3143  
Parent PID: 3142  
  
Executing command: ls  
Desktop      Documents    Music        Public       task1  
Downloads    hello.c     Pictures     task1.c     Videos  
examples.desktop  initial-setup.sh  process.c   Templates  
  
Child process completed.  
samanth@samanth-ubuntu:~$ pwd  
/home/samanth  
samanth@samanth-ubuntu:~$ ./task1  
Enter a Linux command: pwd  
Parent Process  
Parent PID: 3150  
Child PID : 3151  
  
Child Process  
Child PID : 3151  
Parent PID: 3150  
  
Executing command: pwd  
/home/samanth  
Child process completed.  
samanth@samanth-ubuntu:~$
```

This program demonstrates process creation using `fork()`, command execution using `execlp()`, and process synchronization using `wait()`. It also shows how parent and child processes execute independently while sharing a parent-child relationship.

Task-2:

```

45 root      20  0      0      0      0 S  0.0  0.0  0:00.00 oom_reaper
47 root      0 -20     0      0      0 I  0.0  0.0  0:00.00 kworker/R-writeback
48 root      20  0      0      0      0 S  0.0  0.0  0:00.88 kcompactd0
49 root      25  5      0      0      0 S  0.0  0.0  0:00.00 ksmtd

samanth-ubuntu@samanth-ubuntu:~$ uname -a
Linux samanth-ubuntu 7.0.0-29-generic #29-Ubuntu SMP PREEMPT_DYNAMIC Sat Jul 18 01:12:25 UTC 2026 aarch64 GNU/Linux
samanth-ubuntu@samanth-ubuntu:~$ lscpu
Architecture: aarch64
CPU op-mode(s): 64-bit
Byte Order: Little Endian
CPU(s): 4
On-line CPU(s) list: 0-3
Vendor ID: Apple
Model name: 
Model: 0
Thread(s) per core: 1
Core(s) per socket: 4
Socket(s): 1
Stepping: 0x0
BogoMIPS: 48.00
Flags: fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics fphp asimdhp cpuid asimdrdm jscvt fcma lrcpc dcpop sha3 asimddp sha512 asimdfhm
dit uscat ilrcpc flagm sb paca pacg dcpodp flagm2 frint bfi16 bti16 afp

NUMA:
NUMA node(s): 1
NUMA node0 CPU(s): 0-3

Vulnerabilities:
Gather data sampling: Not affected
Ghostwrite: Not affected
Indirect target selection: Not affected
Itlb multihit: Not affected
L1tf: Not affected
Mds: Not affected
Meltdown: Not affected
Mmio stale data: Not affected
Old microcode: Not affected
Reg file data sampling: Not affected
Retbleed: Not affected
Spec rstack overflow: Not affected
Spec store bypass: vulnerable
Spectre v1: Mitigation: __user pointer sanitization
Spectre v2: Mitigation: CSV2, but not BHB
Srbds: Not affected
Tsa: Not affected
Tsx async abort: Not affected
Vmscape: Not affected

samanth-ubuntu@samanth-ubuntu:~$ ps
PID TTY          TIME CMD
 8768 tty1      00:00:00 bash
 9021 tty1      00:00:00 ps

samanth-ubuntu@samanth-ubuntu:~$ top_

```

```

top - 11:27:23 up 3:15, 1 user, load average: 0.09, 0.11, 0.09
Tasks: 127 total, 1 running, 126 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.1 sy, 0.0 ni, 99.9 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3380.7 total, 2351.8 free, 396.1 used, 787.6 buff/cache
MiB Swap: 3892.0 total, 3892.0 free, 0.0 used, 2984.6 avail Mem

  PID USER      PR  NI   VIRT   RES    SHR  S  %CPU  %MEM    TIME+  COMMAND
 9031 samanth+ 20   0  10312  5320   3256  R   0.3   0.2   0:00.01 top
    1 root      20   0  25868 16292 10556  S   0.0   0.5   0:01.13 systemd
    2 root      20   0      0      0      0  S   0.0   0.0   0:00.03 kthreadd
    3 root      20   0      0      0      0  S   0.0   0.0   0:00.00 pool_workqueue_release
    4 root      0 -20     0      0      0  I   0.0   0.0   0:00.00 kworker/R-rcu_gp
    5 root      0 -20     0      0      0  I   0.0   0.0   0:00.00 kworker/R-rcu_wq
    6 root      0 -20     0      0      0  I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
    7 root      0 -20     0      0      0  I   0.0   0.0   0:00.00 kworker/R-slab_flushwq
    8 root      0 -20     0      0      0  I   0.0   0.0   0:00.00 kworker/R-netns
   10 root      0 -20     0      0      0  I   0.0   0.0   0:00.00 kworker/0:0H-kblockd
   13 root      0 -20     0      0      0  I   0.0   0.0   0:00.00 kworker/R-nm_percpu_wq
   14 root      20   0      0      0      0  S   0.0   0.0   0:00.00 ksoftirqd/0
   15 root      20   0      0      0      0  I   0.0   0.0   0:00.31 rcu_preempt
   16 root      20   0      0      0      0  S   0.0   0.0   0:00.00 rcu_exp_par_gp_kthread_worker/0
   17 root      20   0      0      0      0  S   0.0   0.0   0:00.01 rcu_exp_gp_kthread_worker
   18 root      rt   0      0      0      0  S   0.0   0.0   0:00.11 migration/0
   19 root     -51   0      0      0      0  S   0.0   0.0   0:00.00 idle_inject/0
   20 root      20   0      0      0      0  S   0.0   0.0   0:00.00 cpuhp/0
   21 root      20   0      0      0      0  S   0.0   0.0   0:00.00 cpuhp/1
   22 root     -51   0      0      0      0  S   0.0   0.0   0:00.00 idle_inject/1
   23 root      rt   0      0      0      0  S   0.0   0.0   0:00.12 migration/1
   24 root      20   0      0      0      0  S   0.0   0.0   0:00.01 ksoftirqd/1
   26 root      0 -20     0      0      0  I   0.0   0.0   0:00.00 kworker/1:0H-kblockd
   27 root      20   0      0      0      0  S   0.0   0.0   0:00.00 cpuhp/2
   28 root     -51   0      0      0      0  S   0.0   0.0   0:00.00 idle_inject/2
   29 root      rt   0      0      0      0  S   0.0   0.0   0:00.14 migration/2
   30 root      20   0      0      0      0  S   0.0   0.0   0:00.02 ksoftirqd/2
   32 root      0 -20     0      0      0  I   0.0   0.0   0:00.00 kworker/2:0H-kblockd
   33 root      20   0      0      0      0  S   0.0   0.0   0:00.00 cpuhp/3
   34 root     -51   0      0      0      0  S   0.0   0.0   0:00.00 idle_inject/3
   35 root      rt   0      0      0      0  S   0.0   0.0   0:00.11 migration/3
   36 root      20   0      0      0      0  S   0.0   0.0   0:00.02 ksoftirqd/3
   38 root      0 -20     0      0      0  I   0.0   0.0   0:00.00 kworker/3:0H-kblockd
   39 root      20   0      0      0      0  S   0.0   0.0   0:00.00 kdevtmpfs
   40 root      0 -20     0      0      0  I   0.0   0.0   0:00.00 kworker/R-inet_frag_wq
   41 root      20   0      0      0      0  I   0.0   0.0   0:00.00 rcu_tasks_kthread
   42 root      20   0      0      0      0  I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
   43 root      20   0      0      0      0  S   0.0   0.0   0:00.00 kauditd
   44 root      20   0      0      0      0  S   0.0   0.0   0:00.01 khungtaskd
   45 root      20   0      0      0      0  S   0.0   0.0   0:00.00 oom_reaper
   47 root      0 -20     0      0      0  I   0.0   0.0   0:00.00 kworker/R-writeback
   48 root      20   0      0      0      0  S   0.0   0.0   0:00.89 kcompactd0
   49 root      25  5      0      0      0  S   0.0   0.0   0:00.00 ksmtd

```

The experiment demonstrated how Linux abstracts hardware resources by providing system services that efficiently manage CPU, memory, storage, and I/O devices. The terminal commands helped analyze system information, processor details, and running processes in real time.